

Git status, commit e push

Transcrição

Link citado aos 11:45 do vídeo: [documentação do GitHub para geração de chave SSH. \(https://docs.github.com/pt/enterprise-server@3.2/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent\)](https://docs.github.com/pt/enterprise-server@3.2/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent)

Paulo: Nós fizemos muitas das edições lá no site do GitHub, já percebemos vendo o log aqui que no Git, a grande sacada do Linus Torvalds teve quando criou o Git foi que esse repositório não é centralizado, ele está tanto no GitHub quanto agora, depois que você fez o comando `git clone`, ele está na sua máquina. É um sistema de controle de versão distribuído.



Se um dia deletassem os arquivos no GitHub, você teria todo o repositório na sua máquina e também tem o histórico. Então, é muito comum usarmos via linha de comando no nosso repositório local. Por mais que dê para fazer praticamente tudo via web, precisamos saber alguns comandos principais na linha de comando.

O que vamos aprender a fazer agora é saber se está tudo sincronizado com o nosso repositório, veremos qual é o status, se fizemos alguma mudança do nosso local que ainda não foi enviado para o repositório de origem - não chamamos de "central" chamamos de "origem" - e também vamos aprender a fazer o commit. Veremos que tem dois momentos, tem a hora que você "commita" e tem a hora que você empurra todas as mudanças do nosso repositório aqui da nossa máquina para o GitHub. Vamos ver isso na linha de comando?

Guilherme: Vamos.

Paulo: Tive uma ideia, vamos já ensinar a falar assim: "Pega todas as mudanças que fizeram lá no repositório origem e atualiza o projeto aqui no local". Como fazer isso?

Guilherme: Primeira coisa que vamos fazer é pegar aquele link mais uma vez, clicando em "Code" e copiando a URL do nosso projeto. De volta ao console, vou dar um `git pull`.

Paulo: O `git pull` é uma forma de fazer a atualização. Clonar é só a "primeira vez", agora quero puxar tudo que está lá para atualizar aqui.

Guilherme: Exatamente. Vou escrever `git pull` seguido do link que copiei, pressionei "Enter". E pelo o que ele informou estamos em dia.

Paulo: Então não teve atualização. Se alguém tivesse mexido lá enquanto eu e o Guilherme estamos dando a aula aqui. Seria informado aí que tal arquivo foi atualizado, tal arquivo foi deletado, tal arquivo foi adicionado.

Guilherme: Isso aí. Quem vai fazer essas atualizações somos nós, Paulo. Nós vamos mandar coisas diferentes para o nosso repositório.

Vou abrir o VS Code e vou arrastar a pasta do meu projeto, que está no desktop, para dentro do VS Code, para atualizar alguma parte da nossa aplicação. Temos aqui o nosso `app.js`, com os nossos `console.log`, o `index.html`, que inicialmente só tem o `h1`, e temos o `readme.md`.

O que faremos agora? Diferente do CodePen e outras ferramentas onde o HTML já entende o código CSS que já entende o código JavaScript, aqui não. No HTML eu preciso informar que tem um arquivo JavaScript rodando nesse projeto. Então, aqui na linha 10 vou dar um "Enter" e colocar um `<script src="">` aqui. Esse `script src` vai informar que temos arquivos JavaScript a serem lidos: `<script src="app.js">`.

Agora sim, quando rodarmos essa aplicação o código que colocamos no `app.js` será executado no HTML. Então, a única coisa que fizemos foi linkar o arquivo JavaScript com o arquivo HTML.

Paulo: Vale lembrar que isso tudo é um projeto de mentira, mas estamos mostrando como as coisas acontecem. Você mexe em um arquivo, às vezes mexe em mais de um

arquivo ao mesmo tempo. Nesse aqui só mexemos em um.

Guilherme: Podemos usar o terminal integrado do próprio VS Code para fazer alterações. Ao abrir o terminal do VS Code podemos usar o comando `git status` para ver o que tem de diferente nesse projeto. Quando dou um "Enter" ele vai falar que tem um arquivo que foi modificado, o arquivo modificado foi o index.html. Agora temos um desafio. Precisamos pegar essas modificações que fizemos aqui nesse arquivo. O VS Code até indica com uma linha verde no início da linha que foi modificada.

Paulo: O VS Code pega as coisas do GitHub que estão no local e percebe que você mudou aquela linha.

Guilherme: Tem duas formas para criarmos um commit com base nas modificações que fizemos agora. Podemos ir por esses ícones que estão aqui em cima no menu explorer ou podemos fazer via código. Como já fizemos no VS Code online lá no GitHub, agora eu vou fazer via código no terminal.

Paulo: Vamos direto no terminal ver o `git status`. Ele indicou que tem uma modificação sim. Agora vamos falar que queremos commitar esse arquivo. E somos obrigados a dar uma mensagem igual fizemos da outra vez. O comando que usaremos é o `git commit`.

Guilherme: Isso. Vamos usar o comando `git commit` para criar um commit assim como fizemos para os anteriores. Essa mensagem de commit é muito importante, aqui vamos passar exatamente qual arquivo queremos commitar.

Paulo: Pode até usar ponto se quiser o diretório inteiro, pode separar os arquivos por espaços, tem várias formas. Aqui só vamos commitar a modificação do index. E logo depois você põe uma mensagem.

Guilherme: Para criar essa mensagem, essa etiqueta desse commit, vou colocar, por exemplo, "linkando o app.js com o html": `git commit index.html -m "linkando o app.js com o html"`. Pressiono "Enter" e pronto, ele fez o commit e adicionou essa alteração.

Vou limpar o terminal e rodar o `git status` novamente, dessa vez repare que ele não trouxe nenhuma modificação ele até fala aqui "use o push para publicar seus commits locais", ou seja, podemos pegar todas as modificações que fizemos localmente aqui na nossa máquina e mandar essas modificações lá para o nosso código no GitHub. Vamos fazer isso, Paulo?

Paulo: Vamos. Lembrando que "push" em português significa empurrar. Agora vamos empurrar todos os commits, posso ter feito mais de um, e vamos empurrar para o repositório origem.

Guilherme: Vou colocar aqui no terminal o comando `git push origin main`, porque estamos trabalhando só com a divisão principal, o *main*.

Paulo: No site do GitHub, já podemos ver essa atualização que foi "empurrada" para o repositório origem. Já começamos a ver alguma diferença: Git é a ferramenta e GitHub é o lugar que te ajuda a ter esse repositório origem para todo mundo poder trabalhar nesse projeto. Podemos ver que o último commit no arquivo index foi há 3 minutos.

Quando entramos no arquivo podemos, inclusive, clicar no botão "History", que está localizado à direita, para ver toda a história desse arquivo.

Estamos vendo tudo sendo amarrado aí. Pode ser que você tenha tomado uma mensagem de erro, porque para empurrar, ter o direito de escrever no repositório além de ter sido adicionado como colaborador no repositório, que foi o que o Guilherme fez comigo na outra aula, você precisa ter configurado corretamente os seus parâmetros de Git no seu computador. São alguns parâmetros que ficam num arquivo de configuração, como a sua chave pública. Inclusive, em algum lugar você terá uma chave primária.

Vamos deixar os [links dessa configuração \(https://docs.github.com/pt/enterprise-server@3.2/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent\)](https://docs.github.com/pt/enterprise-server@3.2/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent), esse momento tem umas linhas de comando a mais que você tem que fazer, basicamente, só uma vez na vida. Que é gerar sua chave para o GitHub saber, por exemplo, que o Guilherme que está empurrando isso é o Guilherme que tem o login e senha que ele logou no GitHub. Então isso tem aquele mecanismo de chave pública e chave privada de criptografia. Você não precisa entender desse assunto,

futuramente no decorrer da sua carreira é importante, mas é uma forma de você saber que quem está conversando é uma pessoa que pode sim escrever no repositório.

Por que não aconteceu isso antes na web? Porque na web já estávamos logados dentro do GitHub, dessa vez fizemos do nosso local para a nuvem. Nessa hora que empurramos da nossa máquina para a nuvem o GitHub verifica se temos as credenciais para editar esse repositório. Ele checa mecanismos de permissão do Git. Através da chave, de assinatura, etc. Provavelmente muitos de vocês já fizeram isso quando instalaram o Git que está lá no nosso manual.

Guilherme: Isso aí. É por questões de segurança da nossa aplicação mesmo. E na sequência, vamos aprender como navegar entre versões diferentes. Por exemplo, subi uma versão mas quero voltar para uma versão anterior. Ou o que acontece geralmente na época da Black Friday, a maioria dos e-commerces alteram o layout e depois voltam para o layout padrão.

Como fazer essa navegação entre projetos? Será que o pessoal tira tudo na mão mesmo ou eles usam um comando no GitHub para mudar a versão do projeto? É isso que vamos

